

0) Background & Motivation

The reality of GPU training

- **GPU vs CPU memory:** GPUs are fast but have relatively small memory (e.g., 32 GB vs 256 GB CPU memory).
- **Implication:** full dataset usually cannot fit into GPU memory.
- **Solution:** feed the GPU with a **batch of data** each time.
- This is called **Mini-batch Gradient Descent (GD)** — in practice, people usually call it **SGD**.

Terminology

- **Epoch:** one full pass through all training data.
- **Batch size:** number of data points fed to GPU **per iteration**.
- Example: training data = 6400, batch size = 64 → **100 iterations per epoch**.
- **Iteration update** (per batch):

$$w_{ij}^{(k)} = w_{ij}^{(k)} - \Delta w_{ij}^{(k)}, \quad \Delta w_{ij}^{(k)} = \alpha \frac{\partial \text{Div}(Y_B, d_B)}{\partial w_{ij}^{(k)}}.$$

Accumulate Gradient

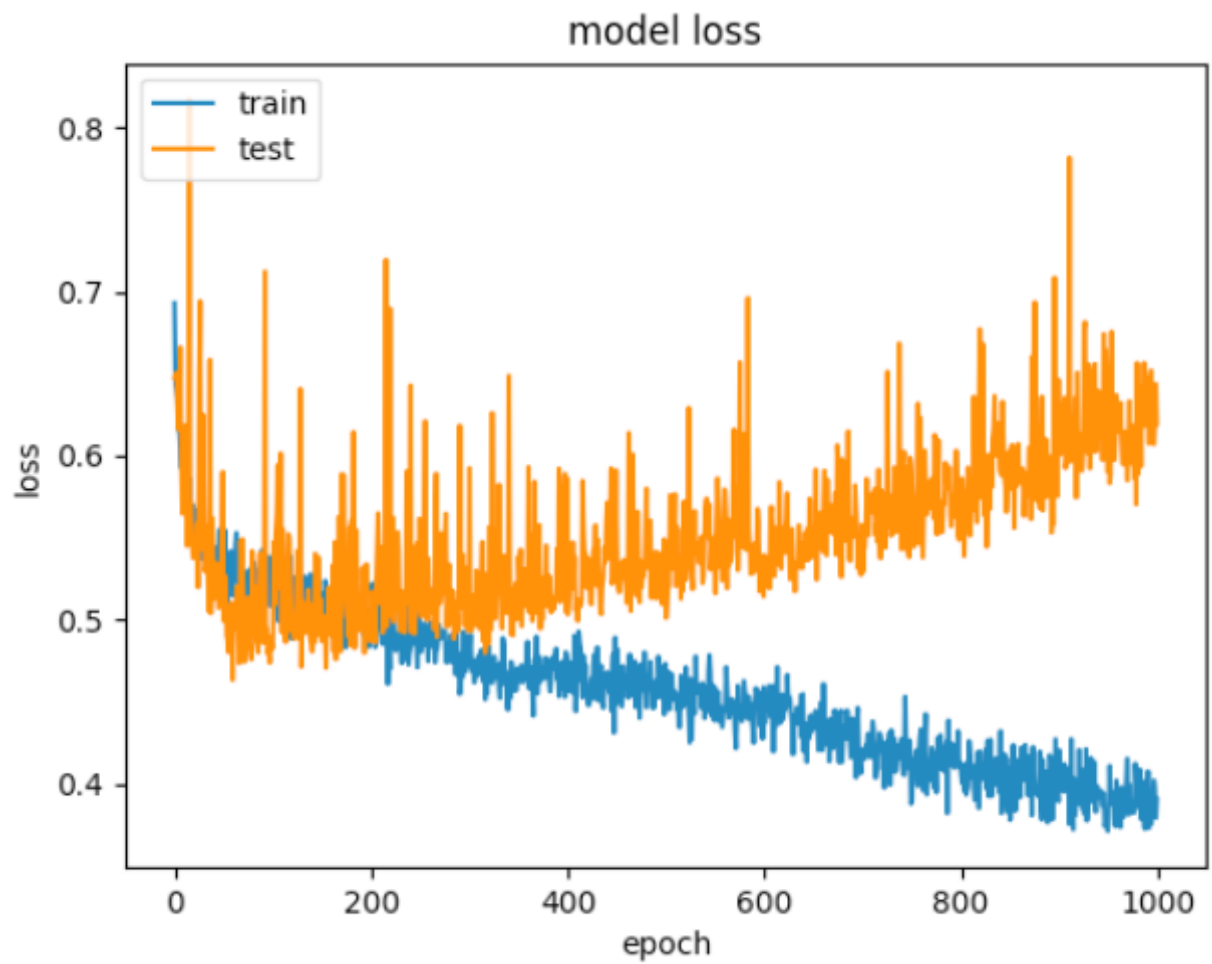
- **Problem:**
 - Model too large,
 - Batch too small → training becomes unstable.
- **Fix:** accumulate gradients over multiple iterations before updating.
 - For example, accumulate for 10 iterations:

$$\Delta w_{ij}^{(k)} + = \alpha \frac{\partial \text{Div}(Y_{B_t}, d_{B_t})}{\partial w_{ij}^{(k)}}, \quad t = 1, \dots, 10,$$

then update once:

$$w_{ij}^{(k)} = w_{ij}^{(k)} - \Delta w_{ij}^{(k)}.$$

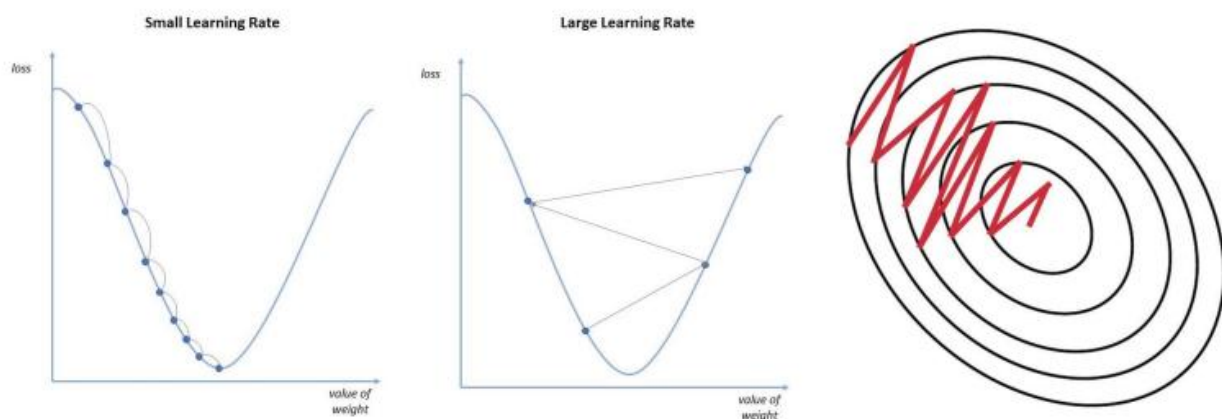
- **Effect:** similar to training with a larger batch size but without needing the same GPU memory.



Warm-up update:

$$w_t = w_{t-1} - \alpha \partial_{w_{t-1}} L.$$

Small $\alpha \Rightarrow$ slow; large $\alpha \Rightarrow$ oscillation.



1) Momentum

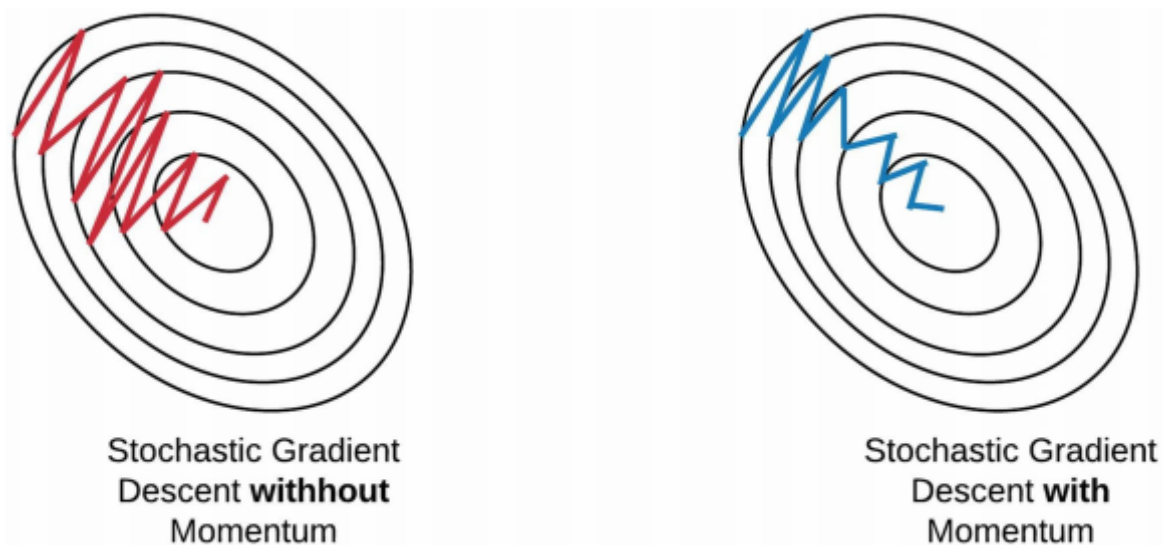
Idea. Reduce oscillation; remember the previous step.

Formula:

$$w_t = w_{t-1} - \alpha \partial_{w_{t-1}} L + \beta \Delta w_{t-1}.$$

Typical values: $\alpha \approx 10^{-3}$, $\beta \approx 0.9$.

Expanding Δw recursively shows momentum is a **decaying average of past gradients**, explaining its effect of accelerating in consistent directions while dampening oscillations.



2) Nesterov's Accelerated Gradient (Nesterov's Momentum)

Gradient computed at the point after a "look-ahead":

$$w_t = w_{t-1} - \alpha \partial_{(w_{t-1} + \beta \Delta w_{t-1})} L + \beta \Delta w_{t-1}.$$

$$(\Delta w_{t-1} = w_{t-1} - w_{t-2})$$

Intuitively, the **look-ahead** allows the gradient to "see" the direction in which momentum will take the parameters, explaining why it is often more stable than standard momentum, especially in steep ravines.

3) From Observation to Per-Component Scaling

- **Observation:** In oscillatory directions, total movement is larger (with or without Momentum).
- **Improvement:** For components with high movement, **reduce** step size; for components with low movement, **increase** step size; scale components **inversely** to their recent total movement.

This reflects a **diagonal preconditioning** approach: using the historical momentum/energy of each coordinate to rescale, thereby alleviating anisotropic curvature differences.

4) Scale by the Inverse Mean-Squared Derivative

Derivation & running average:

$$\begin{aligned}w_t &= w_{t-1} - \alpha \partial_{w_{t-1}} L, \\ \text{New: } w_t &= w_{t-1} - \alpha \frac{1}{E[(\partial_w L)^2]_{t-1}} \partial_{w_{t-1}} L, \\ E[(\partial_w L)^2]_{t-1} &= \gamma E[(\partial_w L)^2]_{t-2} + (1 - \gamma)(\partial_{w_{t-1}} L)^2, \\ E[(\partial_w L)^2]_0 &= 0, \quad \text{use } E[(\partial_w L)^2]_{t-1} + \varepsilon \text{ to avoid zero.}\end{aligned}$$

This is similar to **AdaGrad** but uses exponential moving averages (finite memory) instead of “accumulating to infinity” historical squared gradients, preventing the learning rate from decaying too quickly.

5) RMSProp

F formula:

$$\boxed{w_t = w_{t-1} - \alpha \frac{\partial_{w_{t-1}} L}{E[(\partial_w L)^2]_{t-1} + \varepsilon}},$$
$$E[(\partial_w L)^2]_{t-1} = \gamma E[(\partial_w L)^2]_{t-2} + (1 - \gamma)(\partial_{w_{t-1}} L)^2.$$

Typical: $(\gamma = 0.9, \alpha = 10^{-3}, \varepsilon = 10^{-7})$.

RMSProp can be viewed as a **coordinate-adaptive normalization**: large gradient coordinates are “down-weighted,” while small gradient coordinates are “up-weighted.” This is especially stable for **sparse gradients** or in **noisy** scenarios.

6) Put Momentum Back (RMSProp + Momentum)

$$\text{Momentum: } w_t = w_{t-1} - \alpha \partial_{w_{t-1}} L + \beta \Delta w_{t-1},$$

$$\text{RMSProp: } w_t = w_{t-1} - \alpha \frac{\partial_{w_{t-1}} L}{E[(\partial_w L)^2]_{t-1} + \varepsilon}.$$

Combined:

$$w_t = w_{t-1} - \alpha \frac{\partial_{w_{t-1}} L}{E[(\partial_w L)^2]_{t-1} + \varepsilon} + \beta \Delta w_{t-1}.$$

The first moment (momentum) and second moment (RMS scaling) are **complementary**: the former integrates directional information, while the latter integrates scaling information.

7) Unify the Expression: Two Running Averages

- **Momentum as a weighted sum of past gradients**

$$w_t = w_{t-1} - \alpha (\partial_{w_{t-1}} L + \beta \partial_{w_{t-2}} L + \dots + \beta^{t-1} \partial_{w_0} L) + 0.$$

- **Unified update** (maintaining both first and second running averages):

$$w_t = w_{t-1} - \alpha \frac{E[\partial_w L]_{t-1}}{E[(\partial_w L)^2]_{t-1} + \varepsilon}.$$

$$\begin{aligned} E[(\partial_w L)^2]_{t-1} &= \gamma E[(\partial_w L)^2]_{t-2} + (1 - \gamma)(\partial_{w_{t-1}} L)^2, & E[\cdot]_0 &= 0. \\ E[\partial_w L]_{t-1} &= \delta E[\partial_w L]_{t-2} + (1 - \delta) \partial_{w_{t-1}} L, \end{aligned}$$

This step provides a bridge to **Adam** (unifying first and second moments before correcting for bias).
